NASA SPACE APPS CHALLENGE — DEMO BUILD

Astro Health Station

A visual tool that helps astronauts check their own health during a mission and tells them what to do next — an NFC-driven CYD touchscreen paired with a laptop web dashboard.

How it works, how data flows and is stored, and a tour of every screen on both the laptop and the CYD.

Demo with placeholder data · Not medical advice

Contents

1. What Astro Health Station is
2. Hardware and architecture
3. How the NFC link works
4. How data is stored
5. The serial protocol (CYD ↔ laptop)
6. The laptop web app, tab by tab
7. The CYD touchscreen, screen by screen
8. The health rules engine
9. Gamification: points, streaks, badges
10. Mission profiles and the comms delay
11. Testing and the Simulator
12. Appendix: project layout and running it

> This book describes the system as built for this demo. Every health threshold, body of text and mission number in the system is placeholder data, clearly marked as such in the code (`config/*.json`), ready to be swapped for real NASA data later.

CHAPTER 1

What Astro Health Station is

NASA's Human Research Program names five hazards of spaceflight: space radiation, isolation and confinement, distance from Earth, gravity fields (including microgravity), and hostile closed environments. These hazards affect an astronaut's immune system, bone density, heart and blood vessels, nutrition, sleep and mood over the course of a mission.

Astro Health Station is a small, self-contained tool that lets a crew member check their own health every day, in plain language, without needing a doctor on board. It is built from two parts that work together:

- **A web app on a laptop** — a Flask backend and a plain HTML/CSS/JavaScript dashboard, with no build step and no internet connection required at runtime.
- **A CYD (ESP32 "Cheap Yellow Display") with an NFC reader**, connected to the laptop by USB. Crew members tap NFC cards and tags on it and use its touchscreen for quick check-ins.

A crew member taps their card, gets a short menu on the CYD (mood, symptoms, a reaction test, a breathing exercise), and enters a few vitals on the laptop. The system turns all of that into one clear colour: **GREEN** (good), **YELLOW** (check), or **RED** (act now) — with a plain-language reason, what to do about it, and which of the five hazards is responsible. The same three colours show up everywhere: the web dashboard, the CYD's screen, and its LED.

Why everything is fake data

The challenge's official health datasets were not published at the time this was built. Every number in this system — thresholds, mission comms delays, radiation doses, food calories — is invented and marked `PLACEHOLDER` in the config files that hold it. Nothing here should be read as real medical guidance; a footer reminding of this appears on every page.

Two ways to use it

The system runs two ways, with the exact same code path either way:

- **With real hardware** — a real CYD and NFC cards, connected over USB.
- **In Simulation mode** — no hardware at all. A *Simulator* panel on the web dashboard fakes every card tap and CYD event, for development, testing, or as a fallback if the hardware fails during a demo.

CHAPTER 2

Hardware and architecture

Three boards are involved, only one of which the laptop talks to directly:

Board	Role	Connection
PN532 NFC reader	Reads the raw UID off a card or tag when it's tapped.	UART, wired to the ESP32-S3
ESP32-S3 ("the reader")	Polls the PN532, and relays every UID it sees. It never decides anything — it has no idea who anyone is.	ESP-NOW (wireless, no router) to the CYD
CYD (ESP32-2432S028R, "the station")	320×240 touchscreen, RGB LED, speaker. Shows the crew menu, the result colour, reminders, and the SOS button. Relays raw events to the laptop and shows exactly what the laptop tells it.	USB serial to the laptop

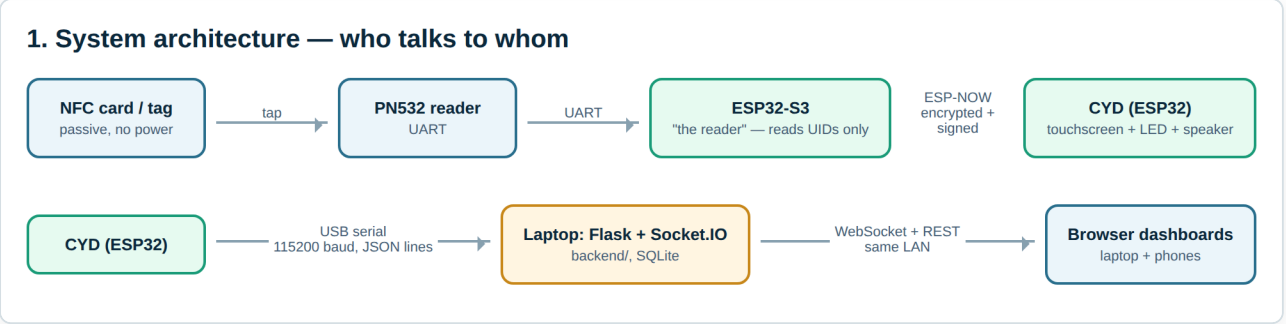


Figure 2.1 — end-to-end signal path, from a tapped card to a browser tab.

The laptop is the source of truth

The CYD only ever reports raw events — "this UID was tapped," "the mood button was pressed," "the SOS button was held." It never decides who is allowed to do what, and it doesn't store anyone's name. All of that — the tag registry, astronaut names, every reading, every rule — lives in the laptop's database and config files. This means the CYD's firmware never needs to change when a threshold or a crew member's name changes.

Why reuse the two-board NFC link?

This hardware started life as a different project (an NFC door lock, "Lifeline"), where a separate ESP32-S3 board reads the PN532 over UART and relays tag UIDs to the CYD over ESP-NOW, rather than wiring the reader straight to the CYD. Astro Health Station keeps that exact link (same boards, same keys, same protocol) because it was already built, wired and working — the CYD firmware just treats every relayed UID as a "scan" event instead of a lock/unlock decision.

> Every NFC tag in this system is identified purely by its UID string. Real cards get a colon-separated hex UID (e.g. `9B:25:0F:54`); tags used only in the Simulator panel use short made-up strings (e.g. `T2000001` for the treadmill). The tag registry doesn't care which kind a UID looks like — it's just a lookup key.

CHAPTER 3

How the NFC link works

Two separate links are involved in getting a card tap onto the screen: a wireless one between the two ESP32 boards, and a wired one between the CYD and the laptop.

3.1 — Reader to CYD: ESP-NOW, encrypted and replay-proof

The reader and the CYD exchange one fixed-size struct (`LinkMsg`) per frame, unicast between two fixed MAC addresses, on a fixed Wi-Fi channel — but never actually joining a Wi-Fi network. Every frame is:

- **Encrypted** with ESP-NOW's own PMK/LMK key pair, set up once in `nfc_link.h` , a header shared byte-for-byte between both boards.
- **Signed** with an 8-byte truncated HMAC-SHA256 over the whole frame, so a frame not made with the shared key is dropped even if it somehow got past the encryption.
- **Protected against replay.** The CYD sends a fresh random challenge (a "nonce") once a second. The reader echoes back whatever challenge it last heard. The CYD only accepts a tag UID whose echoed challenge is the current one or the one right before it, then immediately burns both numbers. A recorded frame, replayed later, carries a stale challenge and is silently ignored — the CYD's screen asks for the card to be scanned again.

3.2 — CYD to laptop: plain JSON over USB

Once the CYD accepts a tag, it has no more cryptography to do — it simply writes one line of JSON to its USB serial port, which the laptop's background thread is always reading. The full message catalogue is in Chapter 5.

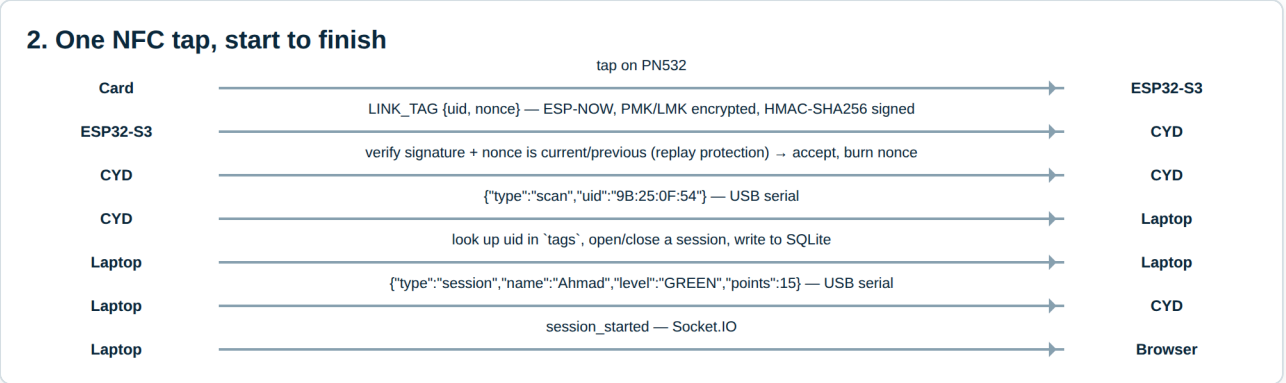


Figure 3.1 — one tap, traced all the way from the card to the browser.

3.3 — What a tag tap means, once it reaches the laptop

Tag type	What happens
Astronaut card	Opens a session for that crew member on the CYD (replacing anyone else's open session). Tapping the same card again ends the session.
Object tag (exercise, sleep, meal, medicine, BP cuff)	Applied to whoever's session is currently open. With no open session, the CYD is told "Tap your card first."
Surgeon card	Unlocks the Flight Surgeon tab on every connected dashboard for 10 minutes.
Emergency card	Starts emergency mode immediately, session or no session.
Unknown UID	The CYD shows "New tag — assign it on the laptop," and the Admin tab pops up an assign dialog. This is how a real card gets connected to a real person, with no code changes.

CHAPTER 4

How data is stored

Everything persists in one SQLite file, `data/astro.db`, created on first run. A single background lock serialises every write, since the serial-reading thread, the reminder scheduler, and ordinary web requests can all try to write at once.

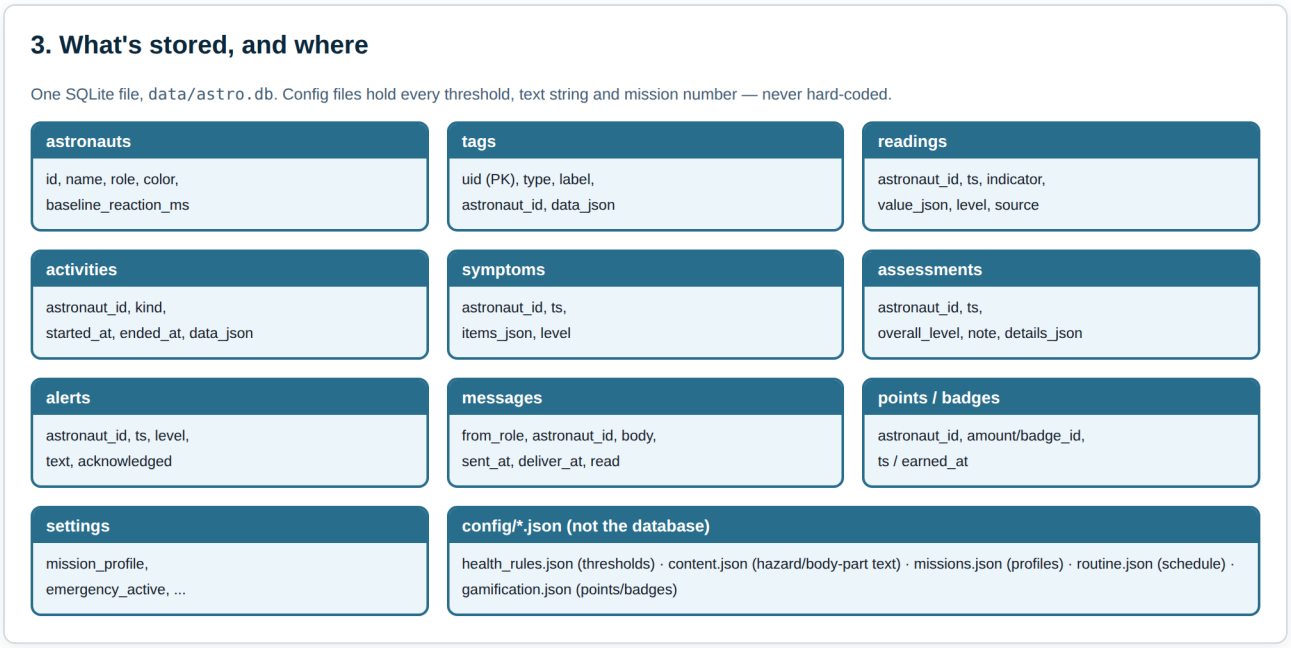


Figure 4.1 — the database tables, and the config files that are deliberately kept out of the database.

Readings vs. activities vs. assessments

Three tables look similar but mean different things:

- **`readings`** is one row per single measurement — a heart rate, a sleep-hours figure, a mood score — each tagged with its own GREEN/YELLOW/RED `level` and a `source` (`manual` typed on the laptop, `cyd` reported by the touchscreen, `tag` from an object tap, or `seed` from the fixed demo history).
- **`activities`** is anything with a start and (eventually) an end — a workout, a night's sleep, a meal, a breathing session — logged by the object tags.
- **`assessments`** is the result of pressing **Assess**: one overall level and note, built by combining that day's readings.

Config files: never a hard-coded number

No threshold, no piece of advice text, and no mission number appears directly in the Python or JavaScript code. They all live in five JSON files under `config/`, every one of them loaded fresh on every read (so editing a file takes effect immediately, no restart needed) and every value inside marked `PLACEHOLDER`:

File	Holds
`health_rules.json`	GREEN/YELLOW/RED ranges for every indicator, the 3-day trend rule, and which symptoms count as a red flag.
`content.json`	Plain-language text: what each hazard means, what each body part does and how space affects it, step-by-step measurement guides, the Learn tab's cards.
`missions.json`	The five mission profiles — gravity, one-way comms delay, main concerns, a placeholder daily radiation dose.
`routine.json`	The daily reminder schedule (morning vitals, exercise blocks, bedtime, ...).
`gamification.json`	Points per action, the four badge rules, the team's weekly exercise-minutes goal.

CHAPTER 5

The serial protocol

One JSON object per line, ending in a newline, over a 115200-baud USB serial connection. Any line that isn't valid JSON (the firmware's own debug prints, for instance) is silently ignored on both ends — this is intentional, not a bug, and lets both sides print plain log text to the same port without confusing the other side's parser.

CYD → laptop

type	Extra fields	When
`hello`	`fw`	On boot
`scan`	`uid`	Any card or tag tapped
`mood`	`uid, value` (1–5)	Mood screen submitted
`symptoms`	`uid, items`	Symptom checklist sent
`reaction`	`uid, trials_ms, avg_ms, lapses, false_starts`	10-trial reaction test finished
`breathing_done`	`uid, seconds`	Breathing exercise finished
`emergency`	`uid` (may be empty)	SOS button held 2s
`cancel_emergency`	—	Cancel button held 3s
`end_session`	`uid`	Done tapped, or 2 minutes idle
`pong`	—	Reply to the laptop's `ping`

Laptop → CYD

type	Extra fields	What the CYD does
`session`	`name, uid, level, points`	Opens the crew menu
`status`	`name, level, note`	Full-colour result banner
`toast`	`text, level`	3-second overlay message
`unknown_tag`	`uid`	"New tag" screen
`exercise` / `sleep`	`event` (start/stop), `minutes` / `hours`	Workout timer, or a "Good night" / "Slept X h" toast
`reminder`	`text`	Beeps, blinks the LED, shows the reminder
`alert`	`level, text`	Yellow or red alert, with beeps
`emergency`	`active, text`	Full emergency mode on/off
`led`	`color`	Sets the RGB LED directly
`idle` / `ping`	—	Back to the idle screen / liveness check, every 5s

> The `level` field is always exactly one of `GREEN`, `YELLOW` or `RED`, kept identical across the database, the web app's colours, and the LED — one vocabulary, everywhere.

CHAPTER 6

The laptop web app, tab by tab

One page, ten tabs, no reloads — everything updates live over a WebSocket. A collapsible Simulator panel sits in the corner of every tab. Screenshots below are from a live run of the actual app, with a real crew member ("Ahmad") checked in.

6.1 — Live

The default tab. Shows "Tap your card" until someone checks in, then their name, a coloured avatar, today's readings as a row of cards, and a vitals form (heart rate, blood pressure, blood sugar, temperature). Sleep, mood, symptoms and reaction time fill in by themselves from CYD events. Pressing **Assess** combines everything into one overall colour, saves it, and sends the result back to the CYD.

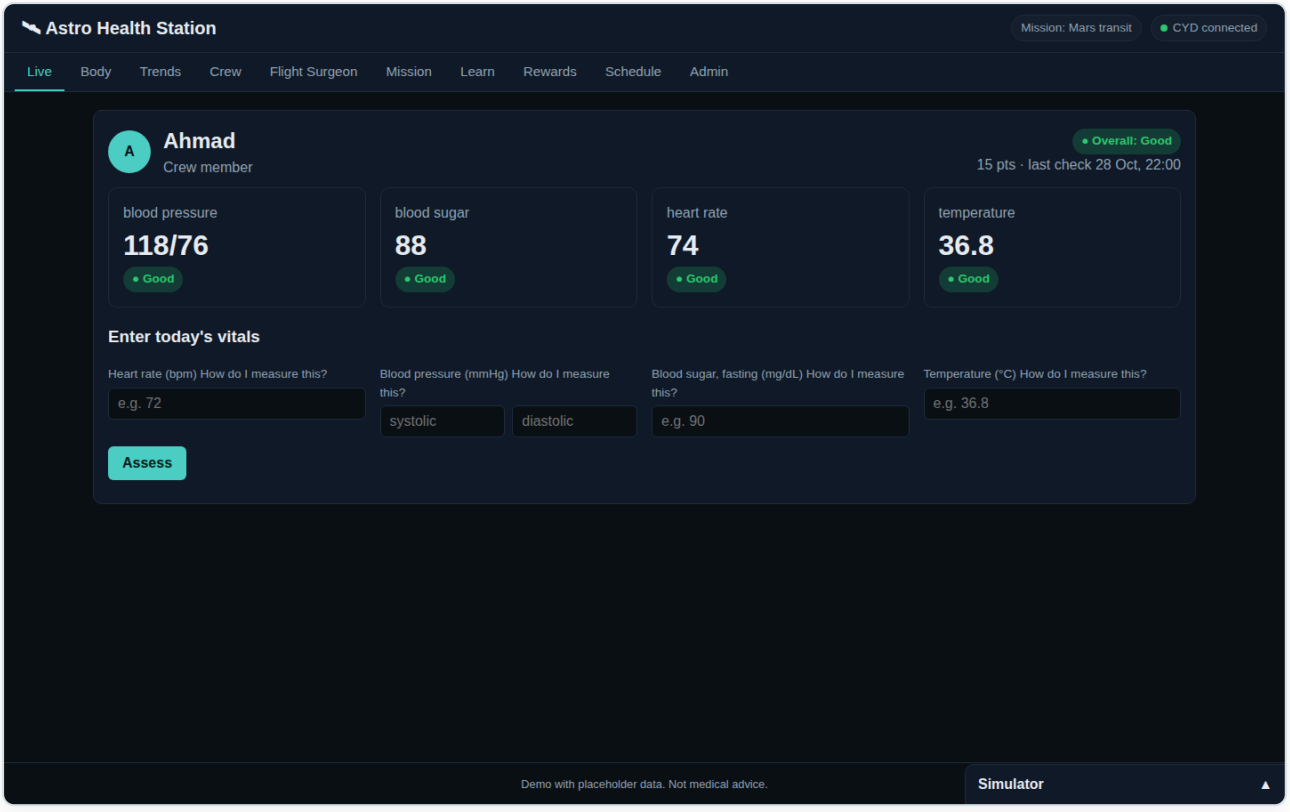


Figure 6.1 — the Live tab after a check-in and an assessment.

6.2 — Body

A hand-drawn, clickable SVG figure — brain, eyes, heart, stomach, skeleton, muscles, and a whole-body glow standing in for the immune system — each coloured by that region's current level, with a gentle pulse on red. Tapping a region opens a panel explaining what it does, how space affects it (tied back to the five hazards), what to do, and a 7-day mini chart of its most relevant indicator.

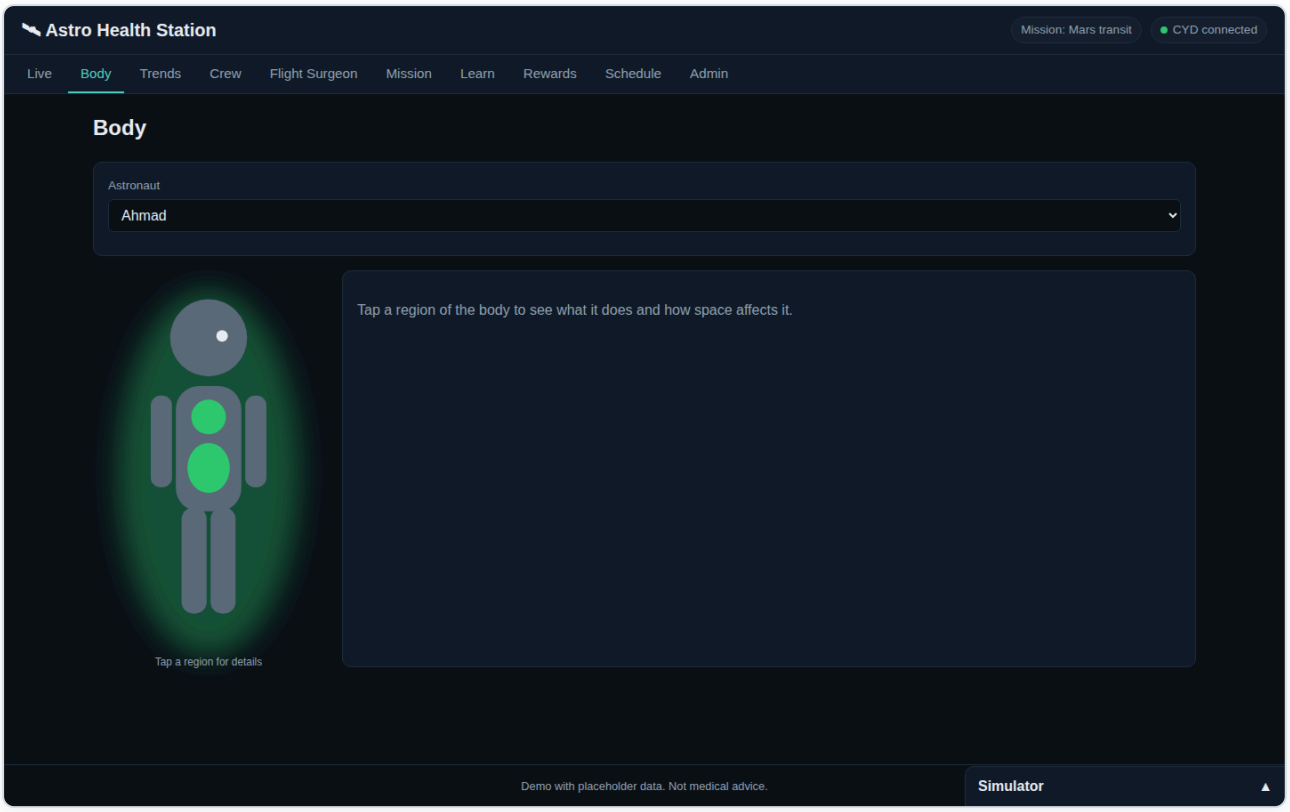


Figure 6.2 — the Body tab. Green regions have in-range readings; grey means no data yet.

6.3 — Trends

Chart.js line and bar charts per indicator, 7 or 30 days back, with green/yellow background bands drawn straight from the threshold config (so changing a threshold changes the chart's bands too). Blood pressure gets two lines (systolic/diastolic); reaction time is tracked against each person's own baseline rather than an absolute number.

6.4 — Crew

Every crew member at a glance: overall colour, when they last checked in, active alerts, and points. Clicking a card jumps to that person in Live, Body and Trends.

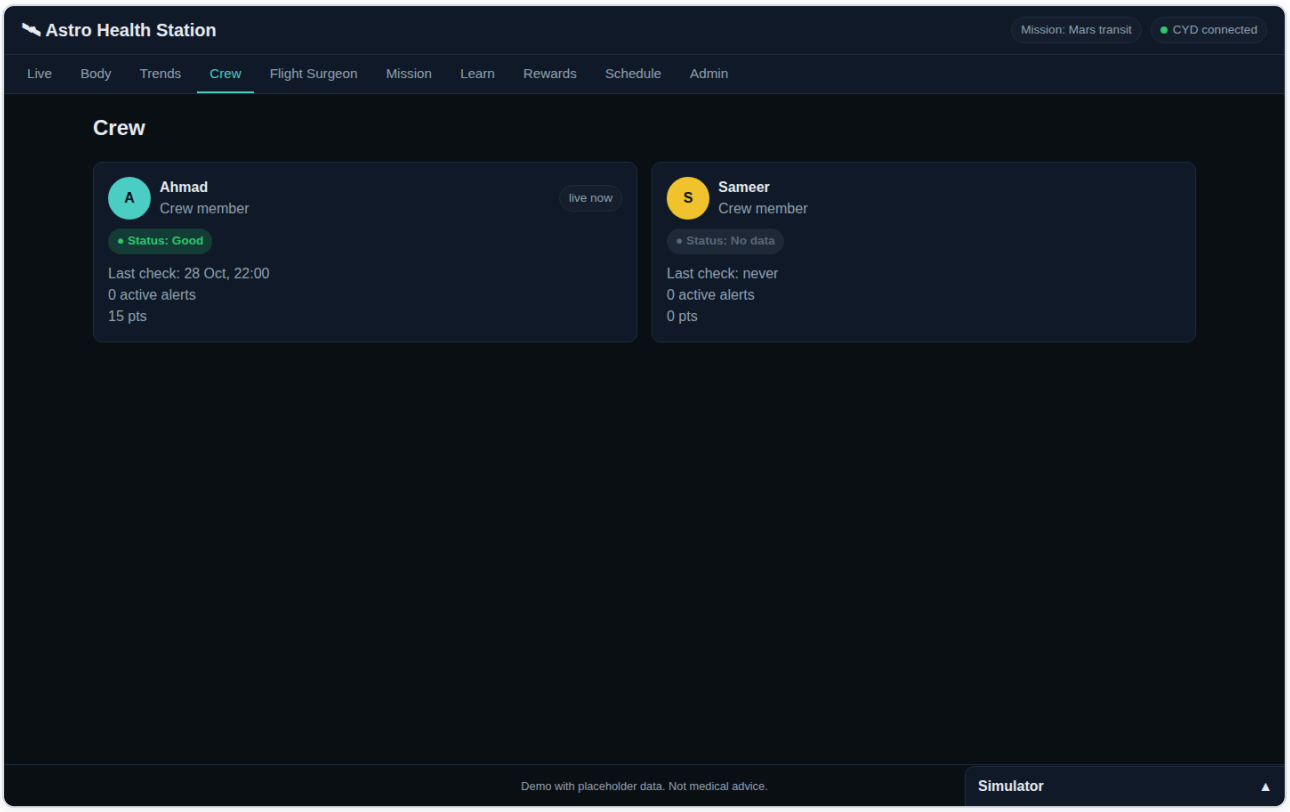


Figure 6.3 — the Crew tab, showing the real roster (Ahmad, Sameer).

6.5 — Flight Surgeon

Unlocked by tapping the surgeon card or entering the demo PIN. Shows the alert inbox and crew reports, and a reply box. Every message to or from Earth waits out the active mission's one-way comms delay (sped up for the demo by a `TIME_SCALE` setting) with a visible countdown; meanwhile the crew sees a self-care checklist instead of a reply. Built to work on a phone, for a surgeon who isn't at the laptop.

6.6 — Mission

Pick a profile — space station, lunar base, Mars transit, Mars surface base, or a rover excursion — and see its gravity, one-way comms delay, placeholder radiation dose, and which of the five hazards matter most there. Switching profiles live-updates the comms delay used everywhere else.

6.7 — Learn

Short, icon-free, plain-language cards: the five hazards, why daily checks matter, what the three colours mean, and a measurement how-to for each vital — all pulled from `content.json`.

6.8 — Rewards

Points, a leaderboard, a team exercise-minutes goal bar, and four badges (*Bone Guardian*, *Sleep Pro*, *Calm Mind*, *Data Hero*), each with a simple rule evaluated from the crew member's own history. A small toast celebrates a new badge the moment it's earned.

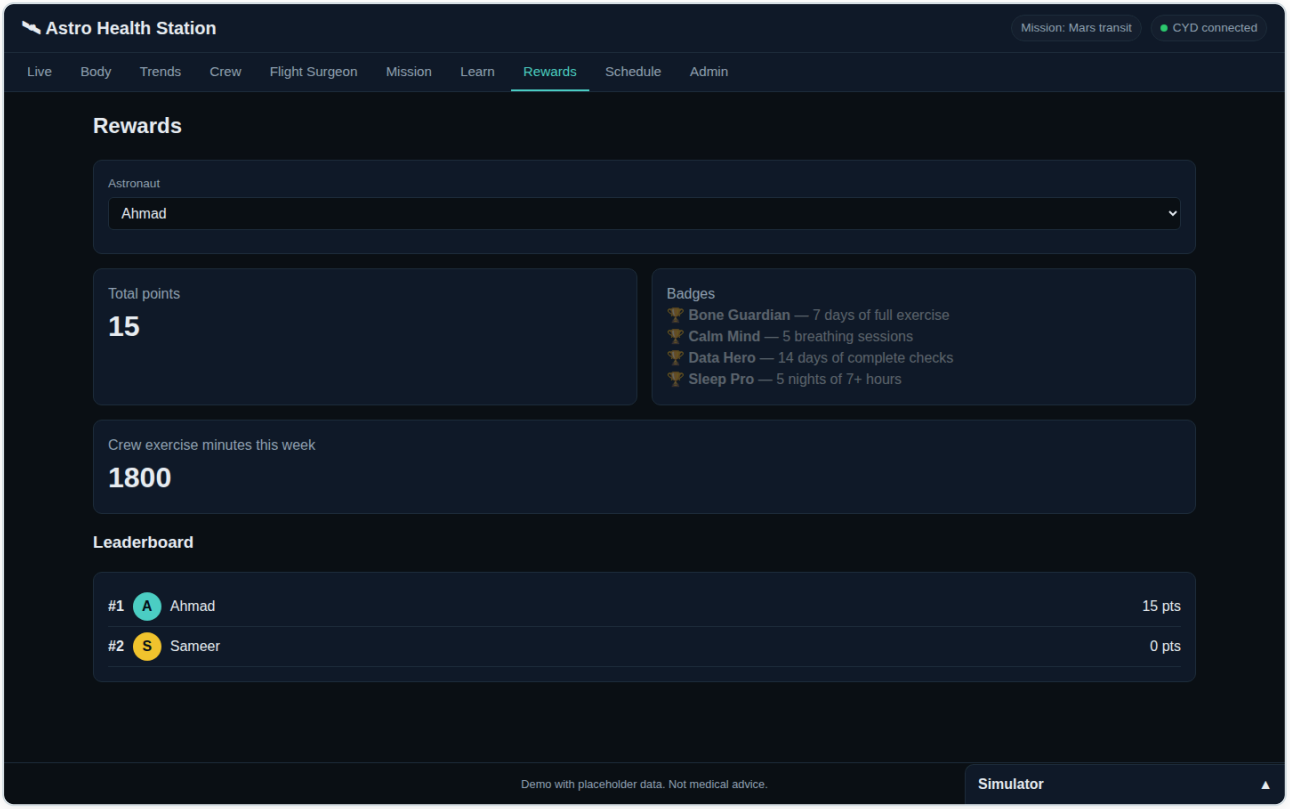


Figure 6.4 — Rewards: points, badge progress and the leaderboard.

6.9 — Schedule

The daily routine from `routine.json`, with a banner when a reminder fires and tasks marked due (yellow) until the matching action is logged.

6.10 — Admin

The tag registry (every UID known to the system, its type, and its label), a rename box for each crew member, a CSV export per person ("Send to Earth"), and the Reset-demo-data button. This is also where an unknown tag — a real NFC card the system has never seen — gets assigned to a crew member or an object type.

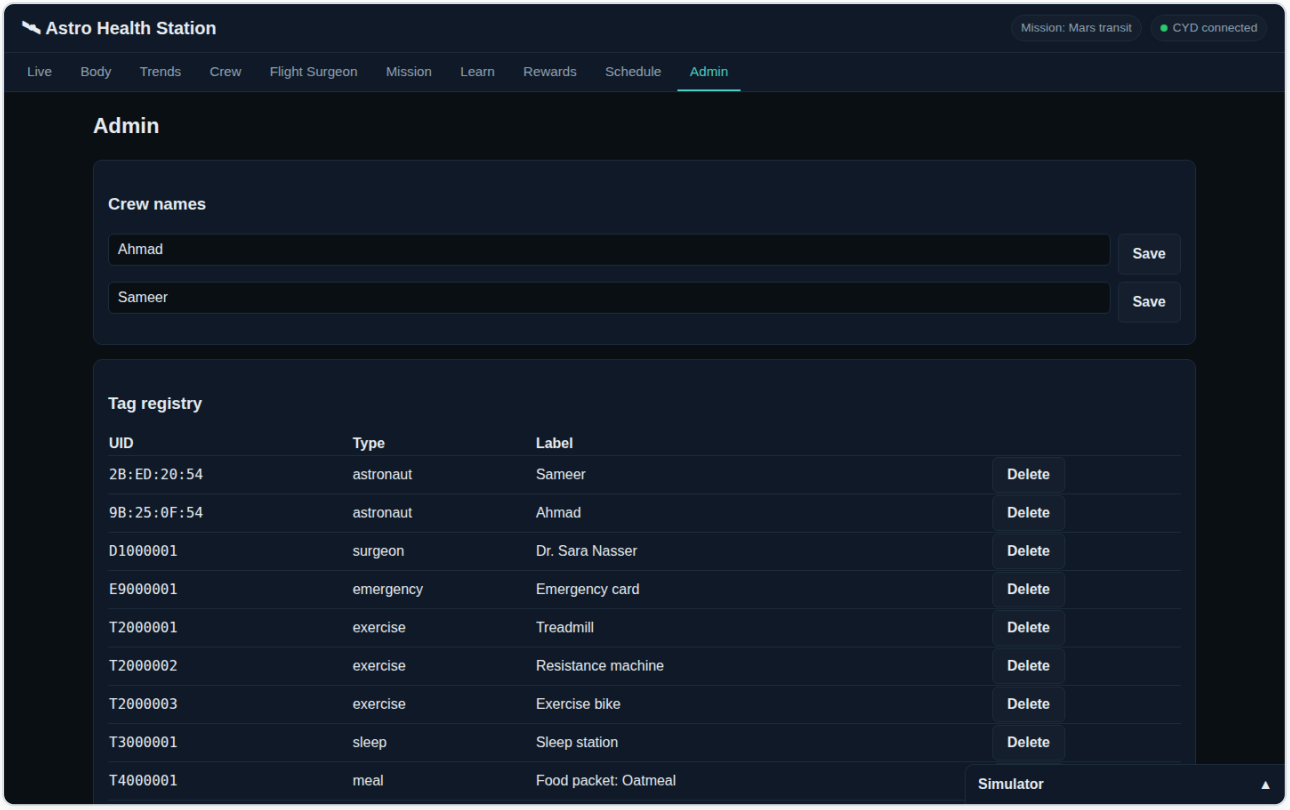


Figure 6.5 — Admin: the live tag registry, showing the real card UIDs for Ahmad and Sameer.

6.11 — The Simulator panel

A collapsible panel, present on every tab, with one button per known card/tag and every fake CYD event (mood, symptoms, a normal or slow reaction test, breathing done, SOS). It calls exactly the same backend code a real card tap would — nothing about the rest of the app can tell the difference. A live, two-way message log sits at the bottom.

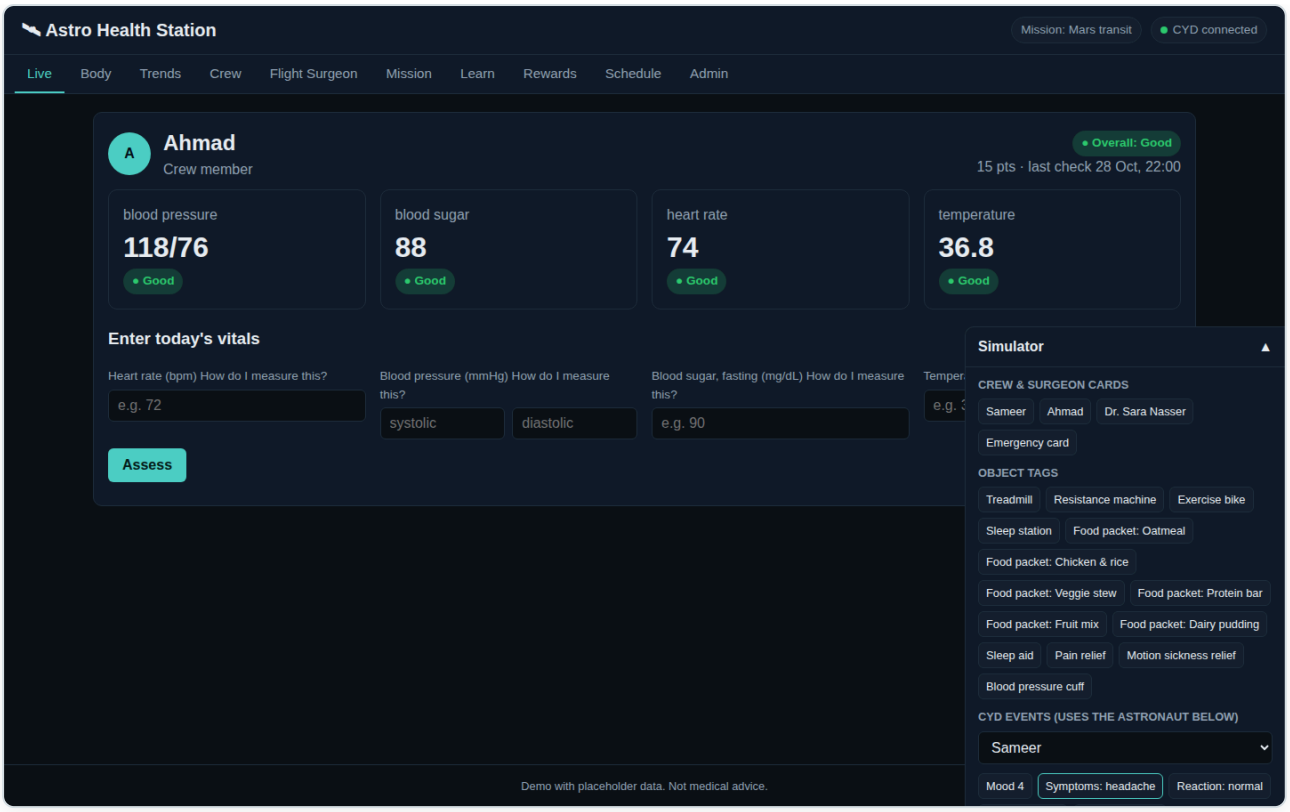


Figure 6.6 — the Simulator panel, expanded over the Live tab.

CHAPTER 7

The CYD touchscreen, screen by screen

Twelve screens, drawn with simple shapes and built-in fonts (no images, no animation library). Touch targets are at least 44 pixels tall. A small red **SOS** button sits in the corner of every screen; holding it 2 seconds starts emergency mode from the hardware alone, even if the laptop is unreachable. The mockups below match the firmware's actual layout and colour palette (the same three status colours as the web app).

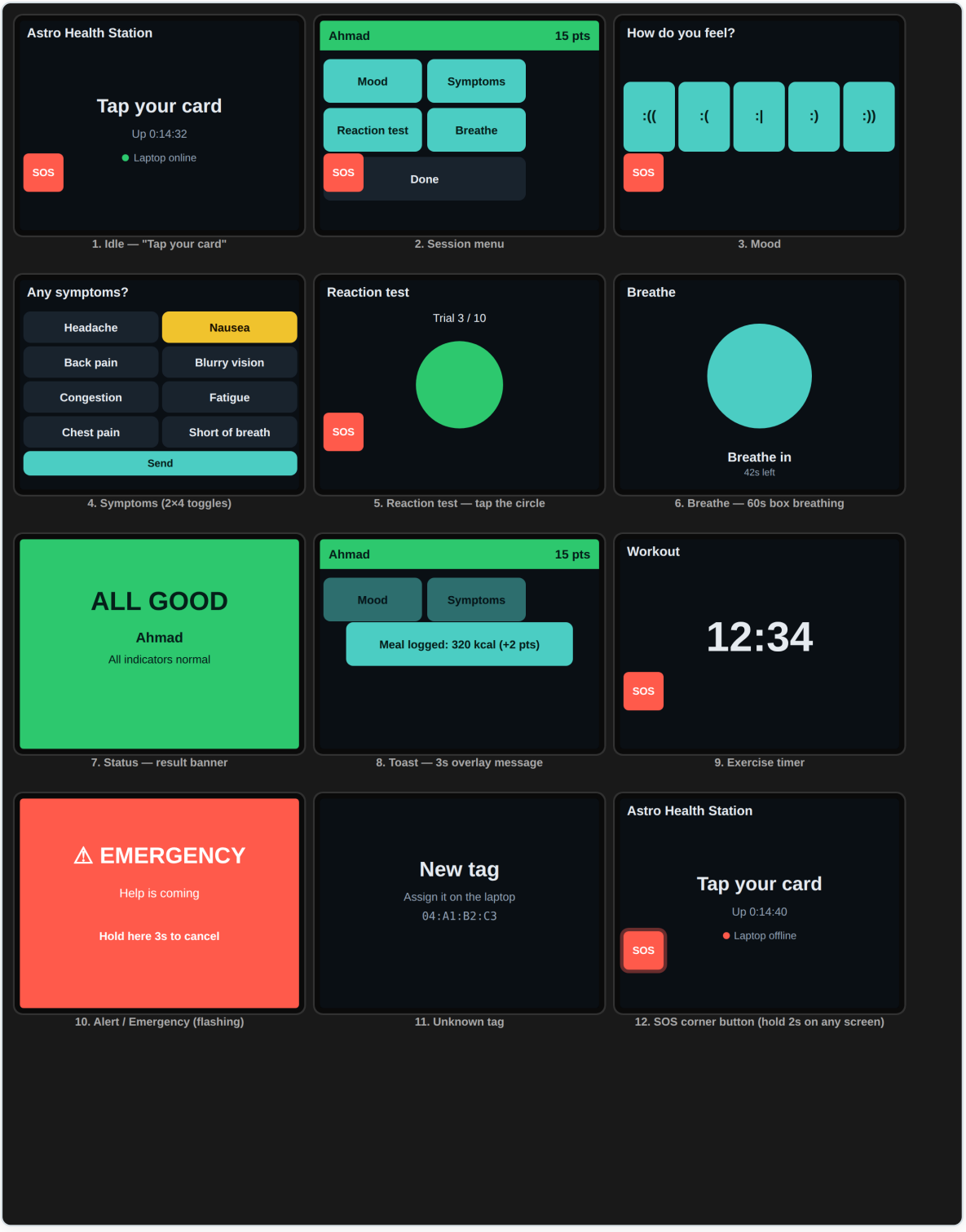
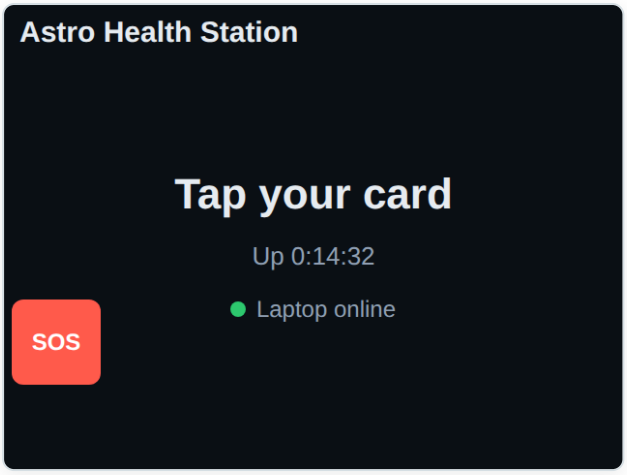


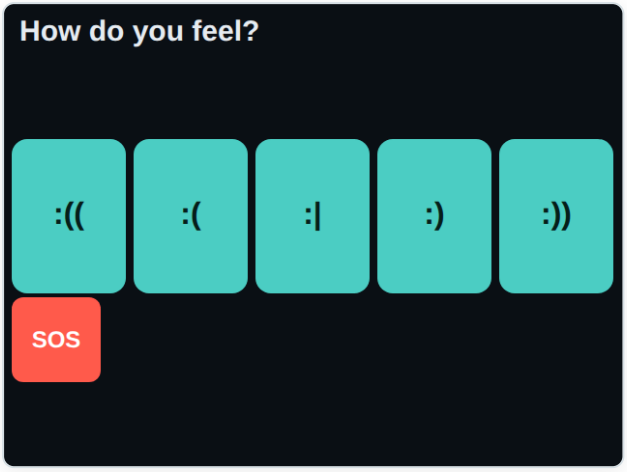
Figure 7.1 — all twelve CYD screens.



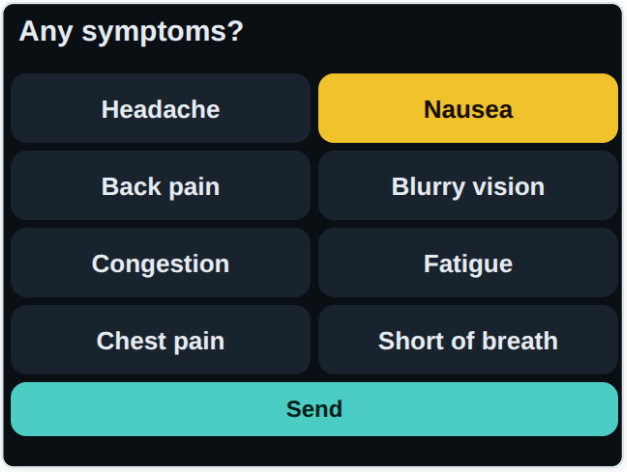
1. Idle. "Tap your card," an uptime clock, and a dot showing whether the laptop has been heard from in the last 10 seconds.



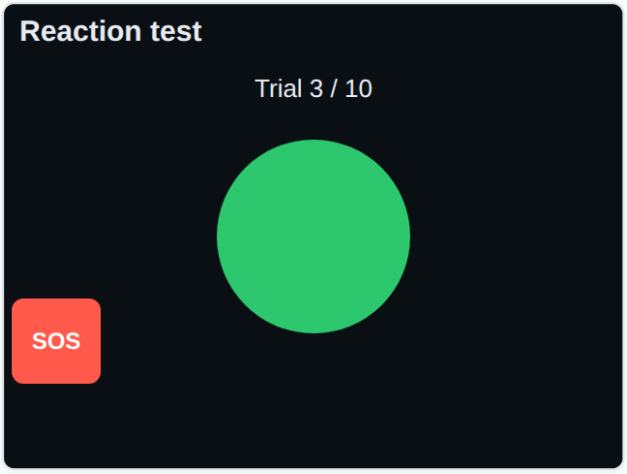
2. Session menu. Opens the moment a card is tapped — optimistically, even before the laptop replies, so it still works if the laptop is briefly unreachable. Colour banner = current level; 2 minutes idle sends it back to Idle on its own.



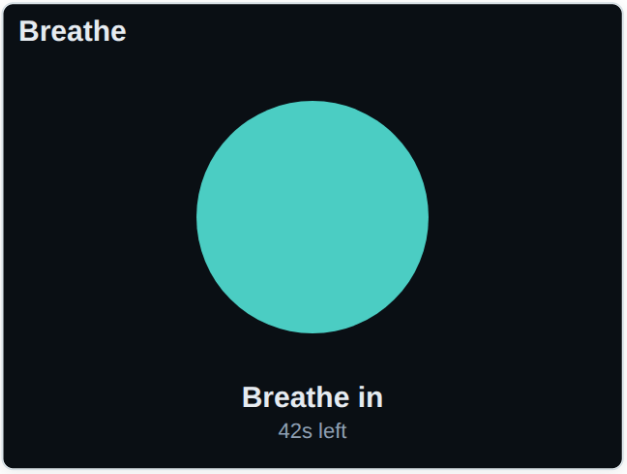
3. Mood. Five big faces, very sad to very happy.



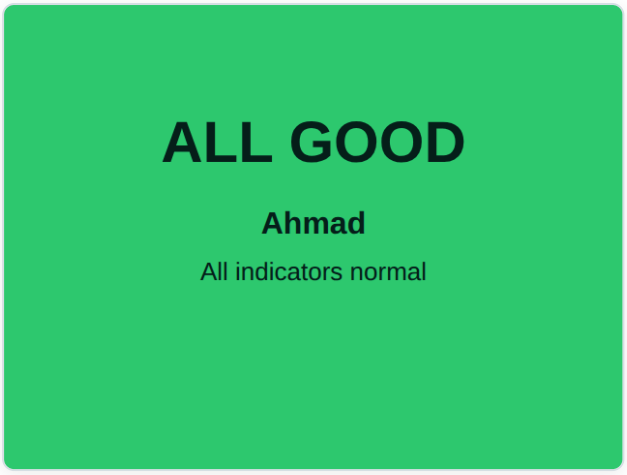
4. Symptoms. An 8-button toggle grid, then Send.



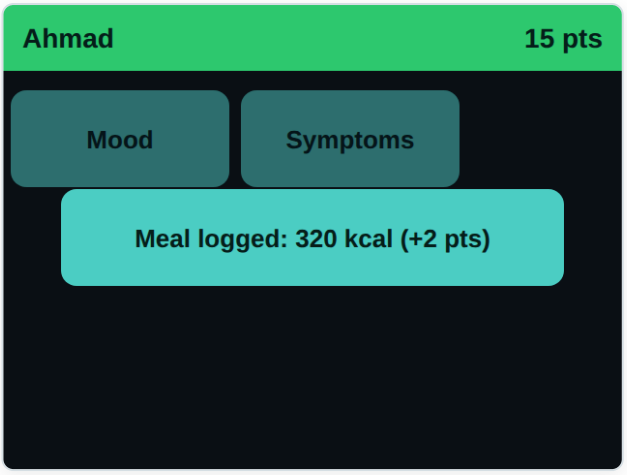
5. Reaction test. 10 trials; a random 2–6s wait, then tap the circle. Too early counts as a false start; over 500ms is a lapse.



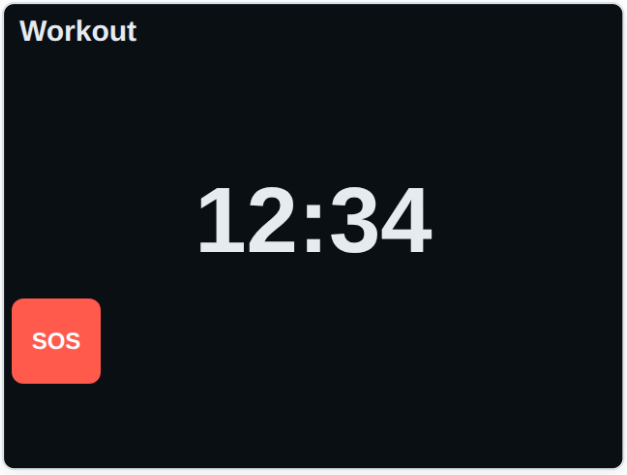
6. Breathe. 60 seconds of box breathing (4s in / hold / out / hold), with a growing and shrinking circle.



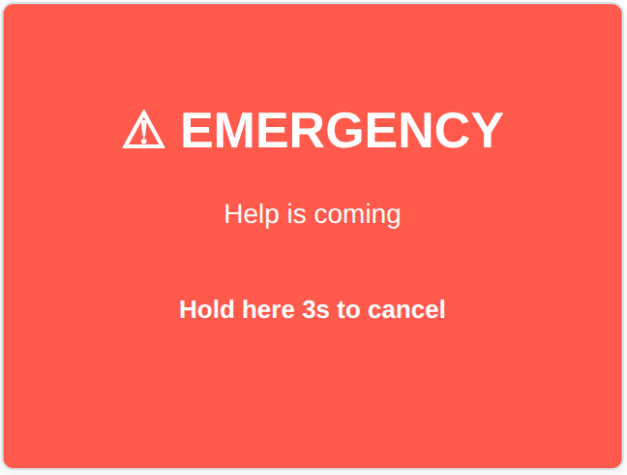
7. Status. The full-colour result banner, sent right after an Assess on the laptop.



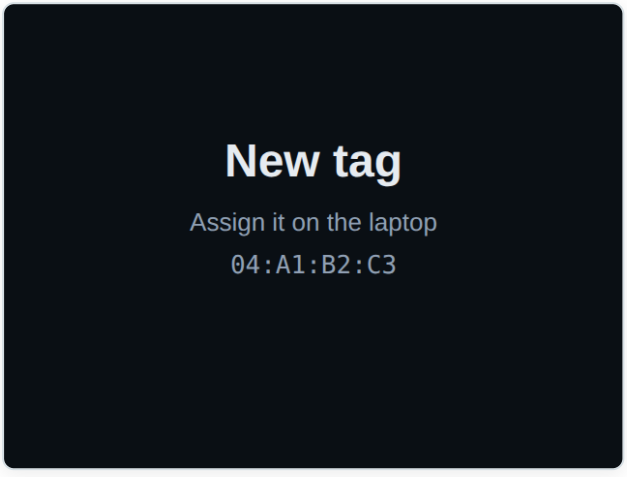
8. Toast. A 3-second overlay for short confirmations, e.g. a logged meal's calories.



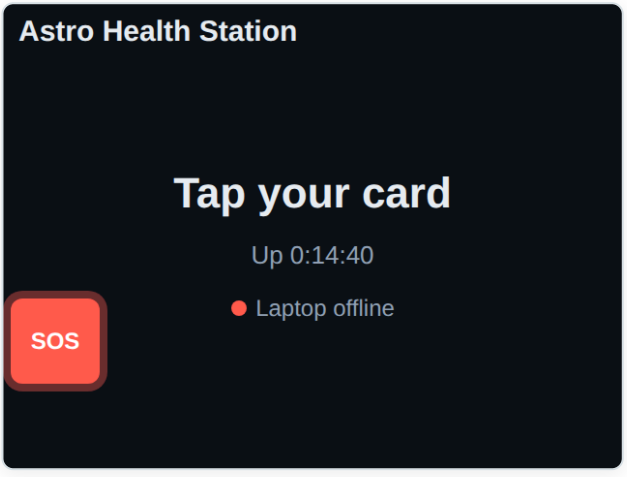
9. Exercise timer. A large running clock while a workout tag is active.



10. Emergency. Flashes, beeps, and only cancels with a deliberate 3-second hold — hard to trigger by accident.



11. Unknown tag. Shown for 3 seconds when a UID the system has never seen is tapped.



12. SOS corner button. Present on every screen; a 2-second hold works even with the laptop disconnected.

CHAPTER 8

The health rules engine

Every reading becomes a level by one small, pure function per indicator shape — no thresholds are ever written directly in the code, only read from `config/health_rules.json`.

Indicator	Unit	GREEN	YELLOW	RED
Heart rate	bpm	60–100	50–59 or 101–120	below 50 or above 120
Blood pressure	mmHg	90–129 / 60–79	130–139 / 80–89	140+ systolic, 90+ diastolic, or systolic below 85
Blood sugar (fasting)	mg/dL	70–99	100–179	below 70 or 180+
Temperature	°C	36.1–37.5	35.5–36.0 or 37.6–38.0	below 35.5 or above 38.0
Sleep	h/night	7+	6–6.9	below 6
Mood	1–5	4–5	3	1–2
Reaction time	vs. baseline	within 10%	10–25% slower	25%+ slower, or 3+ lapses
Exercise	min/day	90+	45–89	below 45
Water	ml/day	2000+	1500–1999	below 1500

Three rules that sit on top of the table

- **Overall level** is always the worst level among that day's indicators — one red reading makes the whole day red.
- **Trend escalation:** if an indicator has gotten worse on each of the last 3 days in a row, it's bumped up one level even if today's raw number looks fine on its own.
- **Symptoms:** any symptom at all is at least YELLOW; blurry vision, chest pain or shortness of breath is RED outright, and triggers the same alert and flight-surgeon report as a RED assessment.

A RED result does three things at once: it alerts every connected dashboard, shows the full-screen alert on the CYD, and queues an automatic report to the flight surgeon — delivered after the active mission's comms delay, just like a message a crew member sent by hand.

CHAPTER 9

Gamification

Every logged action (a vitals check, a finished assessment, a workout, a night's sleep, a meal, a mood check, a symptom report, a reaction test, a breathing session) awards points from `config/gamification.json`. Four badges are recomputed after every assessment:

Bone Guardian — 7 days in a row of a full (90+ minute) exercise day.

Sleep Pro — 5 nights of 7+ hours of sleep.

Calm Mind — 5 completed breathing sessions.

Data Hero — 14 days with at least one completed check.

A small, non-blocking toast celebrates new points or a new badge; a simple leaderboard and a team-wide weekly exercise-minutes goal bar round out the Rewards tab.

CHAPTER 10

Mission profiles and the comms delay

Profile	Gravity	One-way comms delay	Main concerns
Space station (LEO)	0 g	~1 s	Bone/muscle loss, sleep
Lunar base	0.17 g	~1.3 s	Radiation, dust, isolation
Mars transit	0 g	3–22 min	Radiation, isolation, acting alone
Mars surface base	0.38 g	3–22 min	Acting alone, radiation, re-adapting to gravity
Rover excursion	same as its base	same as its base	Effort, hydration, short check-ins

A `TIME_SCALE` setting (default 60×) compresses the comms delay, reminder schedule, and sleep durations for a live demo, without changing any of the underlying logic — the same code runs at real-time speed just by setting it to 1.

CHAPTER 11

Testing and the Simulator

37 automated tests (`pytest`) cover the rules engine, the serial protocol parser, tag-tap session logic, and the REST API, each running against a throwaway SQLite file so they never touch real data.

For everything that needs real hardware to exercise by hand, the Simulator panel calls exactly the same backend function a real card tap or a real CYD event would (`backend/sessions.py` 's `handle_scan()` , and the same message handler the serial bridge uses) — so a feature tested through the Simulator is tested through the real code path, not a separate mock.

> The CYD firmware (clean build, about 32% of flash and 14% of RAM used) has been flashed to and verified on the real board as part of this build: the display, the touchscreen, and a live NFC tap relayed from the real ESP32-S3 reader were all confirmed working. Two real hardware quirks turned up and were fixed along the way — TFT_eSPI needed its pin configuration in a sketch-local `tft_setup.h` file rather than a plain `#define` block (the display library is compiled separately from the sketch, so it never saw the defines), and the display needed the HSPI peripheral rather than TFT_eSPI's default of VSPI to match this board's wiring. Touch calibration may still need a small per-unit tune on different physical hardware.

APPENDIX

Project layout and running it

Path	What's there
`app.py`	Starts Flask + Socket.IO + the serial bridge; `--port`, `--simulate`
`backend/`	Config loader, DB, seed data, serial bridge, protocol, session logic, rules engine, scheduler, comms delay, gamification, REST routes
`config/`	The five placeholder-data JSON files (Chapter 4)
`templates/`, `static/`	The one-page web app: HTML shell, CSS, one JS file per tab, the body SVG, vendored Chart.js and Socket.IO (no CDN at runtime)
`firmware/astro_cyd/`	The CYD sketch, split into tabs: main/state, ESP-NOW receive, serial protocol, the 12 screens, LED/sound
`tests/`	37 pytest tests

Running it

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python app.py              # auto-detects the CYD's USB port
.venv/bin/python app.py --simulate   # no hardware at all
```

Then open `http://localhost:5000`. Full setup, the demo script, and troubleshooting live in the project's own `README.md`.